Сессионный проект

Классификация планктона WHOI-Plankton при ограниченном количестве данных

Ruslan Papina, группа M25-555

Вариант P: классы, в которых от 101 до 200 изображений

Код проекта: https://github.com/riubi/test-classify-img

Цель работы - посмотреть, как ведут себя разные стратегии обучения CNN, когда на класс есть всего 100-200 изображений: своя небольшая свёрточная сеть, обученная с нуля, и ResNet18 с весами ImageNet при разной глубине дообучения (только fc, layer4, layer3-4, вся сеть). Основная метрика сравнения - Macro F1 на test. Весь код, разбиение данных, метрики и графики по каждому запуску лежат в репозитории.

1. Данные и выбор классов

Используется датасет WHOI-Plankton (project-oceania/whoi-plankton на Hugging Face). Это 3 563 596 снимков проточного цитометра Imaging FlowCytobot (IFCB), сделанных у острова Martha's Vineyard в 2006-2014 годах, 103 класса. Классы очень несбалансированы: от 4 изображений (Karenia) до 2.6 млн (mix), см. рис. 1.

По варианту (фамилия Papina, буква P) нужно взять все классы, в которых от 101 до 200 изображений. Число изображений по классам взял из карточки датасета (гистограмма Samples per class для split train, других split'ов в датасете нет). Сумма по гистограмме равна 3 563 596, значит, она посчитана по всему датасету. Таблица со всеми классами сохранена в results/class_counts.csv. В диапазон 101-200 попало 7 классов, всего 1043 изображения (табл. 1).

Доступ к parquet-файлам project-oceania/whoi-plankton выдают вручную. Заявку я отправил, но на момент выполнения работы она была на рассмотрении, поэтому сами картинки выгружал из открытой копии nf-whoi/whoi-plankton. Она собрана из того же архива WHOI (doi:10.1575/1912/7341), но без класса mix и двух самых маленьких классов (Akashiwo - 5 и Karenia - 4 изображения): 3 563 596 - 2 606 720 - 5 - 4 = 956 867, ровно столько изображений в копии. Скрипт prepare_data.py читает из копии только столбец label, находит нужные строки, выгружает только их и проверяет, что по каждому из 7 классов картинок столько же, сколько в официальной гистограмме. Совпало по всем классам. Если доступ к официальному датасету есть, то же самое запускается с параметром --source project-oceania/whoi-plankton.

Таблица 1. Выбранные классы и разбиение

класс	что это	всего	train	val	test
Cerataulina_flagellate	цепочка диатомеи Cerataulina с прикреплёнными жгутиковыми	142	99	22	21
Chaetoceros_flagellate	диатомея Chaetoceros с прикреплёнными жгутиковыми	149	104	22	23
Emiliania_huxleyi	кокколитофорида, мелкая круглая клетка	148	104	22	22
Pleuronema_sp	инфузория Pleuronema, овальная клетка	115	81	17	17
Strombidium_oculatum	инфузория Strombidium, коническая клетка с ресничками	152	106	23	23
Strombidium_wulffi	инфузория Strombidium, коническая клетка с ресничками	157	110	23	24
dino_large1	крупная округлая динофлагеллята	180	126	27	27
ВСЕГО		1043	730	156	157

Разбиение на train / validation / test в пропорции 70 / 15 / 15 стратифицированное: train_test_split из sklearn в два шага (сначала 70/30, потом оставшиеся 30% пополам) с stratify по классу и random_state=42. Получилось 730 / 156 / 157 изображений, доли классов во всех трёх частях одинаковые. Список файлов с пометкой части сохранён в splits/split.csv, и все эксперименты читают разбиение из этого файла, поэтому у всех моделей одни и те же выборки. Во всех запусках фиксирую seed=42 для random, numpy, torch и для генератора DataLoader, который перемешивает train.

Дисбаланс внутри выбранных классов небольшой (от 115 до 180, отношение примерно 1.6), поэтому взвешивание классов в loss не использовал. Но из-за дисбаланса кроме Accuracy считаю Balanced Accuracy и Macro F1.

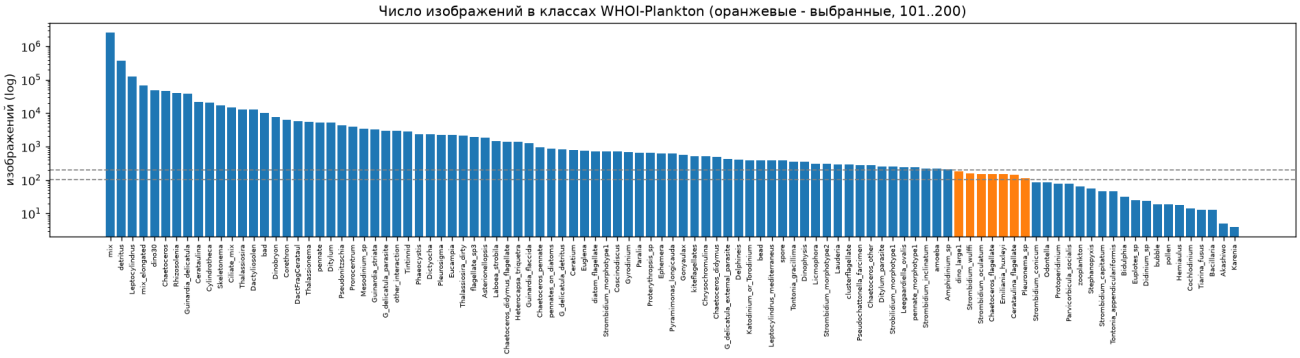


Рис. 1. Число изображений во всех 103 классах (логарифмическая шкала). Оранжевым выделены классы варианта.

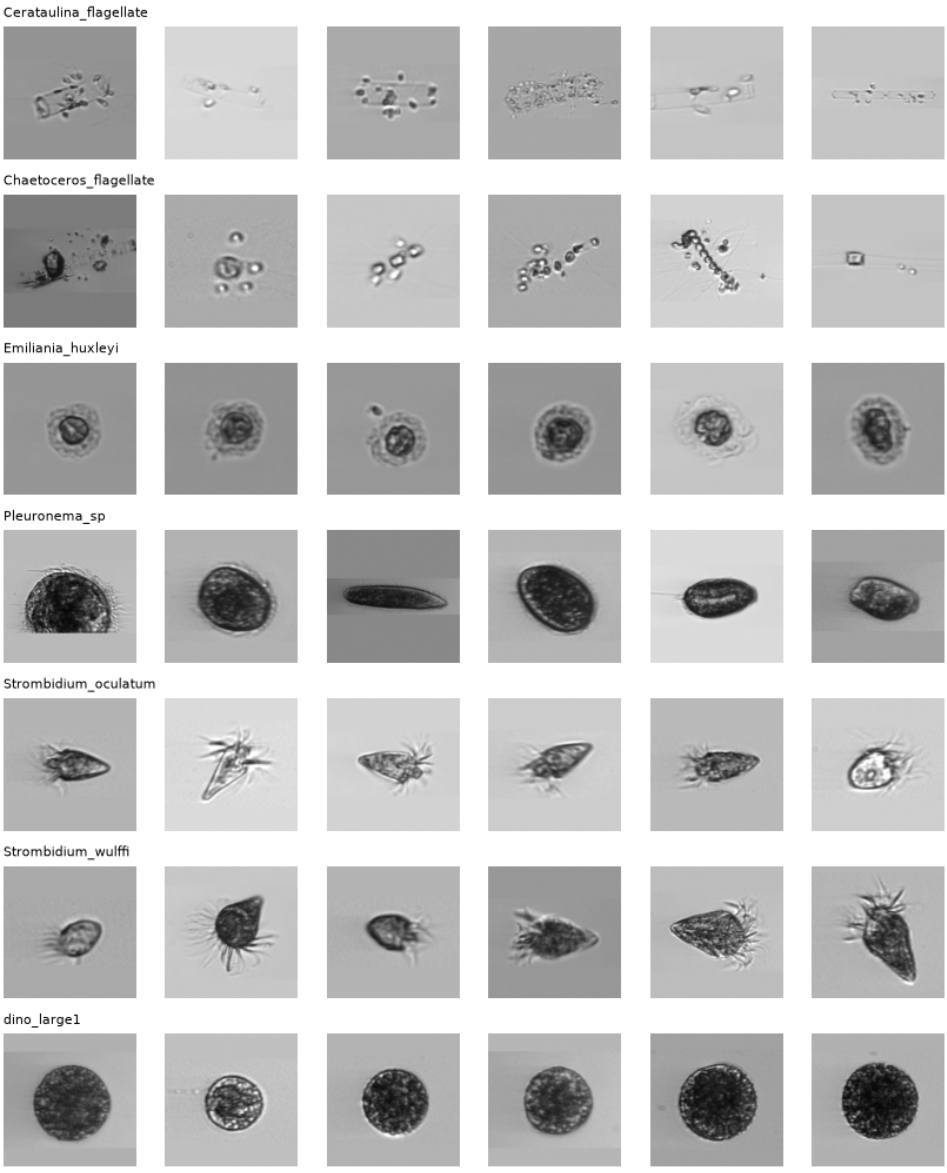


Рис. 2. Примеры изображений выбранных классов после предобработки (паддинг до квадрата и resize 224×224)

2. Предобработка

Снимки IFCB одноканальные (режим L) и все разного размера: ширина от 114 до 1218 пикселей, высота в основном 95-165. Многие объекты сильно вытянуты (цепочки Cerataulina и Chaetoceros), поэтому простой resize до 224×224 сплющил бы их и исказил форму. Предобработка такая:

- открываю изображение в оттенках серого;
- дополняю до квадрата по большей стороне, изображение по центру. Цвет заливки - медиана пикселей по краю снимка: фон у IFCB почти однородный, но у разных снимков разной яркости (видно на рис. 2), и чёрная или белая рамка была бы лишним признаком;
- resize до 224×224 (bicubic);
- Grayscale(num_output_channels=3): канал повторяется три раза, так как и своя CNN (вход 3×224×224 по заданию), и ResNet18 ждут три канала;
- ToTensor: перевод в тензор и в диапазон [0, 1];
- Normalize. Для ResNet18 - mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225], как при обучении на ImageNet, иначе предобученные веса получат вход с другим распределением. Своя CNN учится с нуля, для неё нормализую по статистике своей train-выборки: mean = 0.664, std = 0.164 (одно значение для всех трёх каналов, так как они одинаковые).

Шаги 1-3 делаются один раз при загрузке: картинок всего 1043, поэтому они сразу хранятся в памяти уже в виде 224×224, а аугментации применяются к ним на лету. Для validation и test используются только шаги 4-6, без случайных преобразований.

У такого resize есть минус: после приведения к 224×224 теряется абсолютный размер клетки, мелкая и крупная клетка занимают примерно одинаковую часть кадра. Для IFCB размер - полезный признак (у прибора фиксированное увеличение), но по заданию нужен единый размер 224×224, и для ResNet18 это стандартный вход.

3. Аугментации

Аугментации подбирал исходя из того, как снимает IFCB. Клетки проходят через проточную ячейку, и сначала я считал, что ориентация у них случайная (как оказалось, это не так, см. результаты ниже). Фон и контраст немного отличаются от снимка к снимку, а увеличение у прибора фиксированное. Значит, ориентация и яркость не должны влиять на класс, а форма клетки, её детали (реснички, шипы, прикреплённые жгутиковые) и пропорции - должны. Аугментации применяются только к train.

Таблица 2. Аугментации для train (схема basic)

преобразование	параметры	зачем
поворот на k·90° + малый угол	k = 0..3, доп. угол ±15°, заливка цветом фона	предполагал, что ориентация клетки в потоке случайная. Поворот на 90° для квадрата без потерь. Произвольный угол до 360° не беру: после паддинга длинная цепочка занимает всю сторону и при повороте на 45° у неё обрезаются концы. Углы заливаю фоном, а не чёрным, чёрных треугольников на реальных снимках нет
RandomHorizontalFlip, RandomVerticalFlip	p = 0.5	зеркальное отражение клетки - тот же вид, у организмов в потоке нет «верха» и «низа»
RandomResizedCrop	scale = (0.8, 1.0), ratio = (0.9, 1.1)	небольшое масштабирование и сдвиг. Нижняя граница 0.8, чтобы не отрезать часть клетки (после паддинга объект занимает всю длинную сторону). ratio около 1, чтобы не менять пропорции клетки
ColorJitter	brightness = 0.2, contrast = 0.2	яркость фона и контраст на снимках разные, модель не должна на это опираться

Что не использую и почему

- изменение hue и saturation - снимки серые, цвета нет;
- сильный crop (scale меньше 0.5), Cutout / RandomErasing - у мелких клеток можно вырезать весь объект или его важную часть (например, жгутиковых на цепочке) и получить картинку, которая уже не соответствует метке;
- перспектива, эластичные искажения, сильный shear - форма клетки и расположение ресничек и шипов как раз и отличают классы (например, два вида Strombidium), искажение может сделать из одного класса другой;

- размытие и шум - тонкие детали (реснички, мелкие жгутиковые) и так на пределе разрешения IFCB;
- инверсия цвета - на всех снимках тёмный объект на светлом фоне, в реальных данных инвертированных снимков не бывает.

Для сравнения сделал второй, более агрессивный вариант strong: дополнительный угол ±45°, RandomResizedCrop с scale = (0.5, 1.0) и ratio = (0.75, 1.33), brightness и contrast по 0.4. На рис. 3 видно, что при contrast 0.4 часть снимков почти засвечивается, а при scale 0.5 у цепочек обрезаются концы.

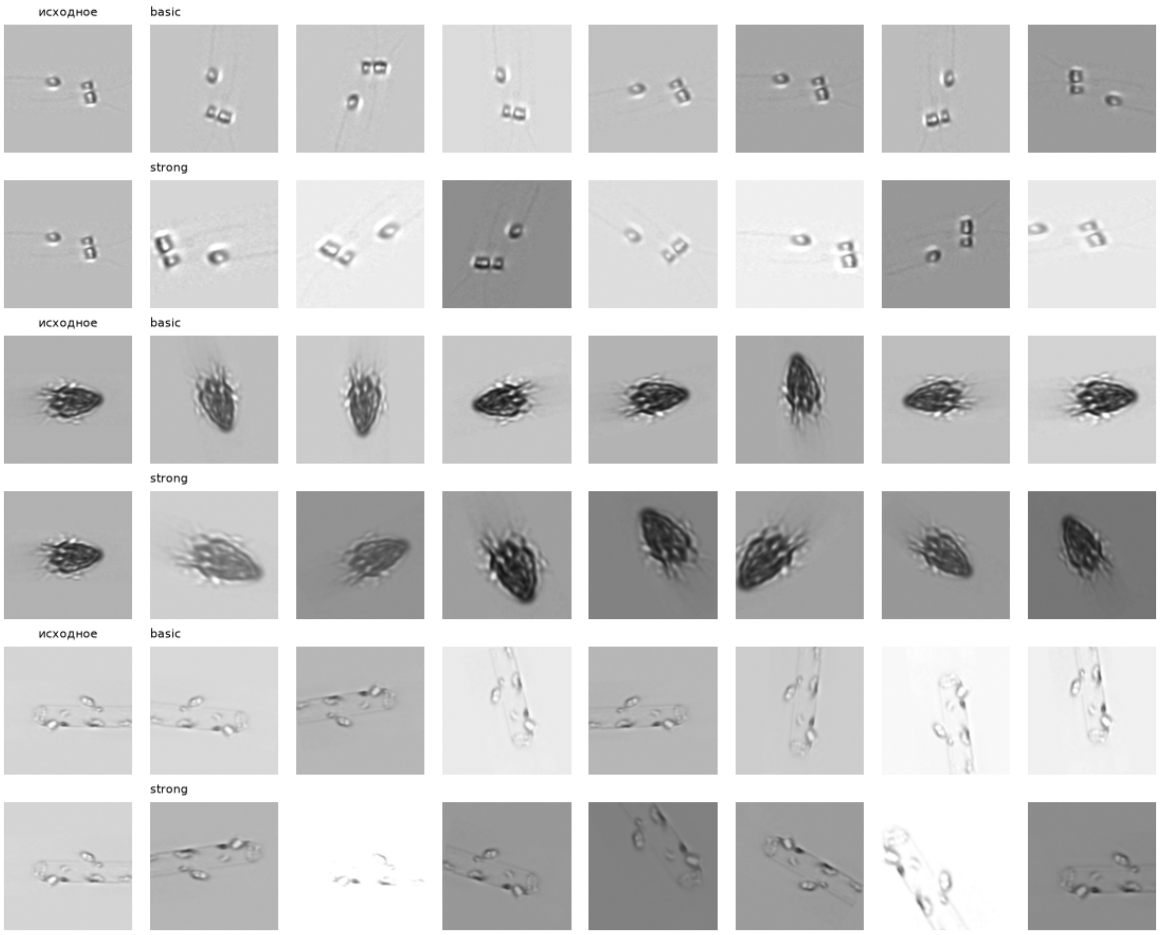


Рис. 3. Исходное изображение и случайные варианты basic и strong

Эксперимент: своя CNN без аугментаций и с аугментациями

Все три запуска с одинаковыми параметрами, меняется только аугментация: своя CNN, ReLU, batch size 32, Adam, lr = 1e-3, dropout 0.5, не больше 50 эпох, early stopping с patience 10 по val macro F1.

запуск	что изменено	best epoch	эпох	train F1*	val F1	test Acc	test macro F1
cnn_noaug	без аугментаций	50	50	0.809	0.782	0.809	0.793
cnn_aug	basic	47	50	0.664	0.634	0.726	0.693
cnn_aug_strong	strong	32	42	0.586	0.595	0.688	0.622

* train F1 здесь и дальше - macro F1 лучшей модели на train, посчитанный без аугментаций и в режиме eval, чтобы его можно было честно сравнить с val и test. Кривые train на графиках считаются во время обучения, с аугментациями и dropout, поэтому они идут ниже.

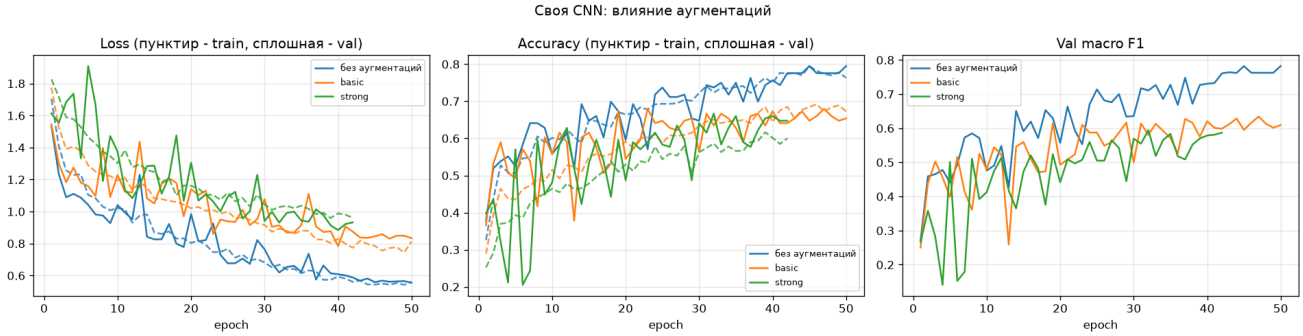


Рис. 4. Кривые обучения своей CNN с разными аугментациями

Результат получился не таким, как я ожидал: лучше всего и по val, и по test сработала CNN без аугментаций (val macro F1 0.782, test 0.793). С basic получилось 0.634 / 0.693, со strong - 0.595 / 0.622.

Переобучения без аугментаций при этом нет: train F1 без аугментаций 0.809 почти равен val 0.782, кривые train и val loss на рис. 4 идут вместе до самого конца, а лучшая эпоха - последняя, 50-я. Сеть скорее недообучена. При 730 картинках и batch size 32 за эпоху делается всего 23 шага оптимизатора, за 50 эпох - около 1150. Аугментации помогают бороться с переобучением, а здесь его нет, поэтому они только усложнили задачу: с basic сеть даже train выучила хуже (train F1 0.664), со strong ещё хуже (0.586). Чем сильнее аугментации, тем больше отставание, strong к тому же остановился по early stopping на 42-й эпохе.

Вторая причина нашлась, когда я посмотрел на размеры исходных снимков: 98.9% изображений шире, чем выше, и даже у круглых dino_large1 отношение ширины к высоте около 1.36. IFCB снимает клетки в потоке, и вытянутые объекты идут вдоль потока, то есть горизонтально, а после паддинга полосы фона всегда оказываются сверху и снизу. Поворот на 90° из схемы basic делает вертикальные объекты и вертикальные полосы, которых в val и test не бывает. Получается, моё исходное предположение о случайной ориентации клеток было неверным.

По правилу задания для дальнейших экспериментов (параметры CNN и ResNet18) беру схему с лучшим val macro F1 среди проверенных, без аугментаций.

Проверка: аугментации без поворотов на 90°

Чтобы проверить догадку про ориентацию, уже после основных экспериментов сделал ещё один вариант, flip: отражения по горизонтали и вертикали (горизонтальную ориентацию они сохраняют), поворот только на небольшой угол ±10° без поворотов на 90°, RandomResizedCrop и ColorJitter как в basic. Остальные параметры те же, что у cnn_noaug.

запуск	что изменено	best epoch	эпох	train F1*	val F1	test Acc	test macro F1
cnn_noaug	без аугментаций	50	50	0.809	0.782	0.809	0.793
cnn_aug	basic	47	50	0.664	0.634	0.726	0.693
cnn_aug_flip	flip	40	50	0.743	0.695	0.790	0.762

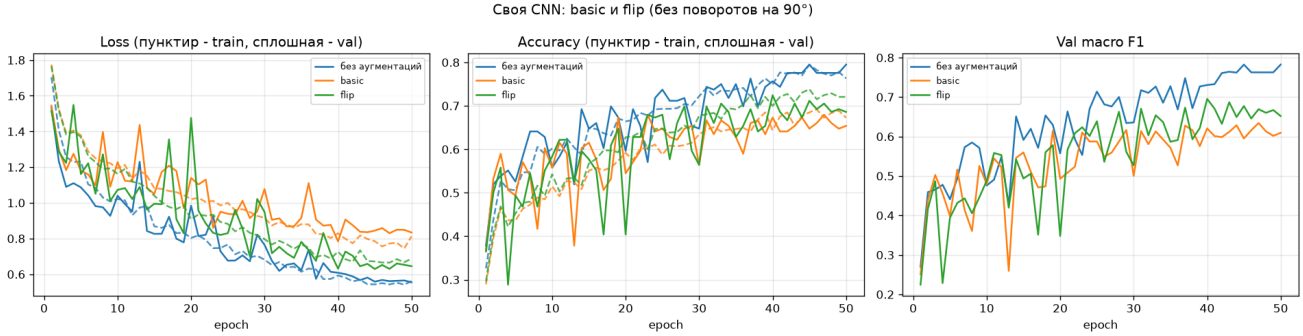


Рис. 4а. Своя CNN: basic и flip

Догадка подтвердилась частично. Без поворотов на 90° качество заметно выросло по сравнению с basic: val macro F1 0.695 против 0.634, test 0.762 против 0.693, train F1 0.743 против 0.664. То есть примерно половина проигрыша basic объясняется неправильной ориентацией. Но и flip всё равно хуже, чем обучение без аугментаций (0.782 / 0.793): вторая причина - недообучение - никуда не делась. За 50 эпох своя CNN не успевает выучить даже исходный train, и любые аугментации ей только мешают.

Основные эксперименты я не переделывал: схема для них была выбрана заранее по val среди трёх исходных вариантов. Но если делать работу заново, я бы начал с flip, а не с basic, и дал бы своей CNN больше эпох.

4. Собственная CNN

Сеть сделана по базовой архитектуре из задания: 4 свёрточных блока Conv2D → BatchNorm → Activation → MaxPooling с 32, 64, 128 и 256 каналами, затем Adaptive Average Pooling → Fully Connected → Activation → Dropout → выходной слой на 7 классов. Свёртки 3×3 с padding 1, MaxPool 2×2, поэтому каждый блок уменьшает картинку в два раза. У свёрток нет bias, так как сразу после них идёт BatchNorm со своим сдвигом. Веса свёрток инициализируются по Kaiming.

блок	слои	размер выхода	параметров
вход		3 × 224 × 224	-
Block 1	Conv 3×3 (3→32) → BN → Act → MaxPool 2	32 × 112 × 112	928
Block 2	Conv 3×3 (32→64) → BN → Act → MaxPool 2	64 × 56 × 56	18 560
Block 3	Conv 3×3 (64→128) → BN → Act → MaxPool 2	128 × 28 × 28	73 984
Block 4	Conv 3×3 (128→256) → BN → Act → MaxPool 2	256 × 14 × 14	295 424
GAP	AdaptiveAvgPool2d(1) + Flatten	256	-
FC	Linear 256→256 → Act → Dropout	256	65 792
выход	Linear 256→7	7	1 799
всего			456 487

Общие настройки обучения для всех моделей: CrossEntropyLoss, не больше 50 эпох, CosineAnnealingLR (lr плавно уменьшается по косинусу от начального значения почти до нуля за 50 эпох), early stopping с patience 10 по val macro F1. После каждой эпохи, если val macro F1 стал лучше, модель сохраняется в checkpoints/. После обучения загружается лучшая по validation модель, и только она оценивается на test. Обучение шло на MacBook Air M2 (MPS).

Сначала вместо косинуса был ReduceLROnPlateau по val macro F1 (lr в 2 раза вниз, если 4 эпохи нет улучшения). Но val - это всего 156 картинок, F1 на нём скачет на несколько процентов от эпохи к эпохе, и lr уменьшался уже на 12-й эпохе, когда сеть ещё явно училась. Сильнее всего это било по запускам с аугментациями, которые сходятся медленнее, поэтому сравнение получалось нечестным. Косинусное расписание от шума на val не зависит, и все результаты в отчёте получены с ним.

Исследование параметров

Базовая конфигурация - запуск cnn_noaug (по итогам раздела 3 аугментации не используются): ReLU, batch size 32, Adam, lr = 1e-3, weight decay 0, dropout 0.5. В каждом следующем запуске меняется ровно один параметр относительно базы. Лучшую конфигурацию выбираю по val macro F1, а не по test, чтобы test не участвовал в выборе.

запуск	что изменено	best epoch	эпох	train F1*	val F1	test Acc	test macro F1
cnn_noaug	база: ReLU, bs 32, Adam, lr 1e-3, dropout 0.5	50	50	0.809	0.782	0.809	0.793
cnn_leaky_relu	activation: LeakyReLU (0.1)	43	50	0.819	0.805	0.822	0.815
cnn_gelu	activation: GELU	43	50	0.869	0.836	0.847	0.846
cnn_bs16	batch size 16	44	50	0.772	0.752	0.777	0.761
cnn_bs64	batch size 64	50	50	0.841	0.802	0.841	0.835
cnn_lr3e-4	lr 3e-4	41	50	0.795	0.805	0.764	0.745
cnn_lr3e-3	lr 3e-3	29	39	0.676	0.706	0.701	0.662
cnn_adamw	AdamW, weight decay 1e-2	45	50	0.815	0.809	0.809	0.799
cnn_sgd	SGD + momentum 0.9 (nesterov), lr 1e-2, wd 5e-4	42	50	0.800	0.742	0.834	0.822
cnn_drop0.2	dropout 0.2	50	50	0.845	0.832	0.834	0.824
cnn_drop0.7	dropout 0.7	43	50	0.774	0.736	0.815	0.807
cnn_5blocks	5 блоков: 32-64-128-256-512	45	50	0.969	0.890	0.885	0.886
cnn_best	5 блоков + GELU + dropout 0.2	17	27	0.867	0.830	0.866	0.863

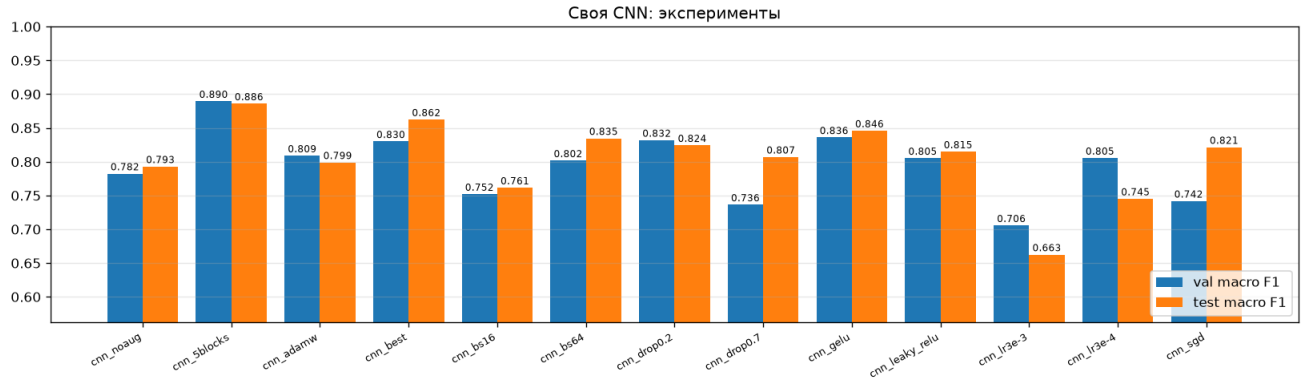


Рис. 5. Macro F1 на val и test для всех запусков своей CNN

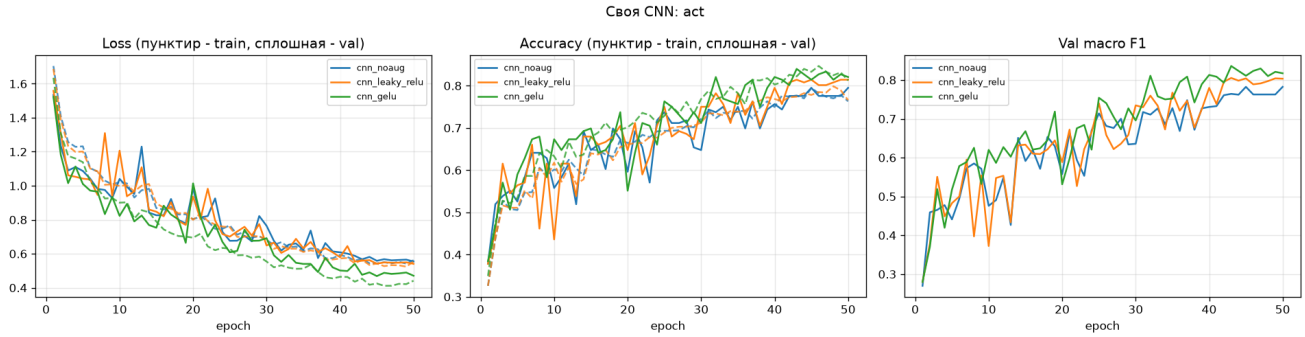


Рис. 5а. Функция активации: ReLU, LeakyReLU, GELU

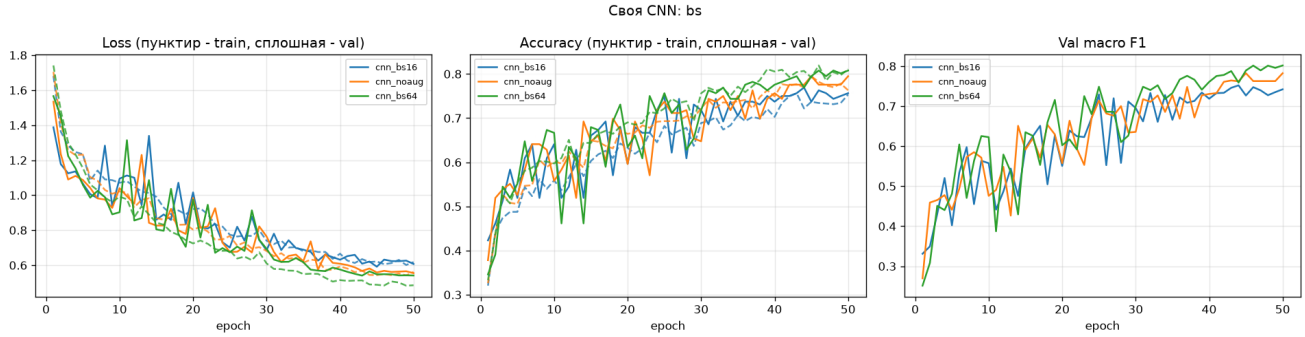


Рис. 5б. Batch size 16 / 32 / 64

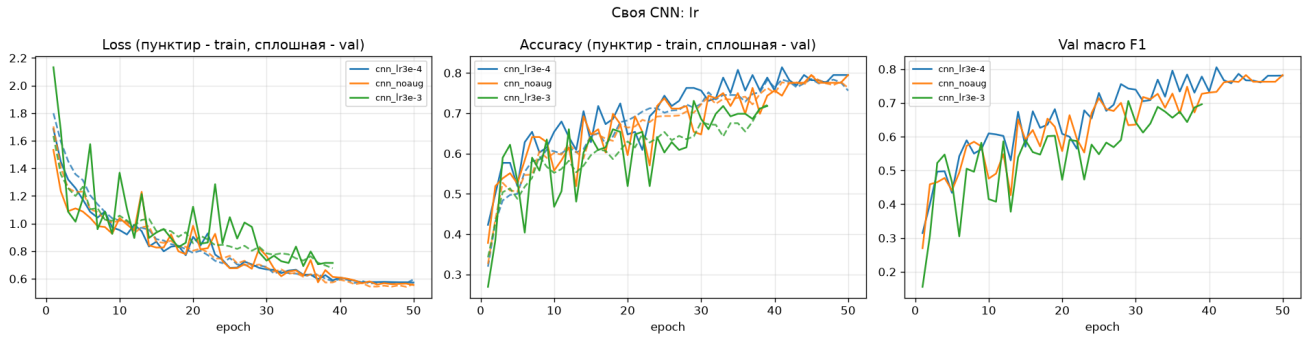


Рис. 5в. Learning rate 3e-4 / 1e-3 / 3e-3

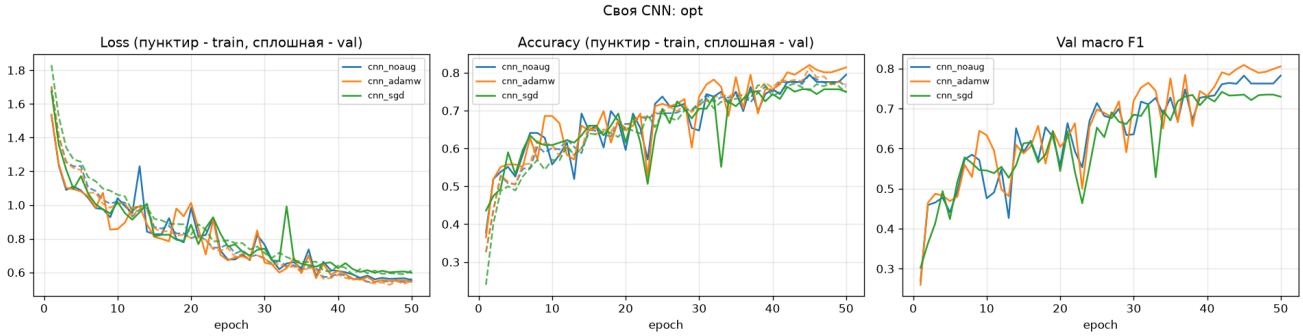


Рис. 5г. Оптимизатор: Adam, AdamW, SGD

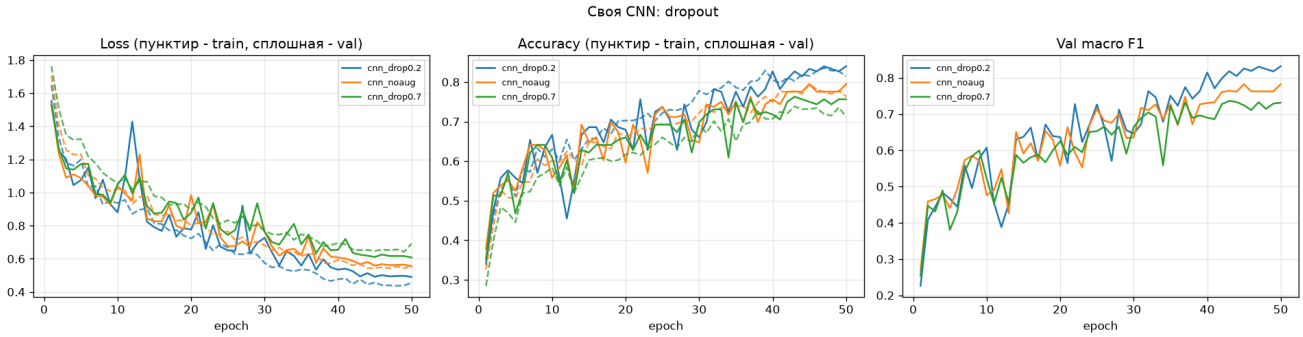


Рис. 5д. Dropout 0.2 / 0.5 / 0.7

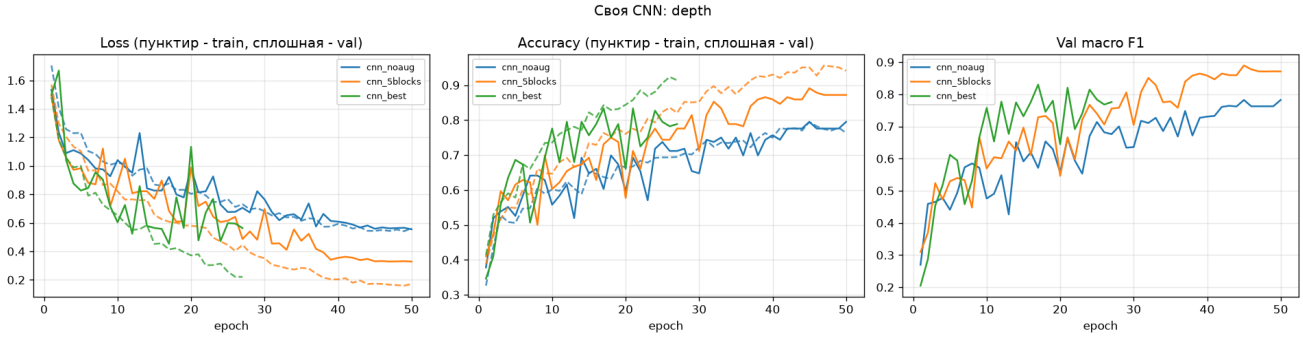


Рис. 5е. 4 и 5 свёрточных блоков, комбинация лучших изменений

Сильнее всего повлияла глубина сети: с пятым блоком (512 каналов) val macro F1 вырос с 0.782 до 0.890, test - с 0.793 до 0.886. Объяснение, по-моему, в рецептивном поле. После 4 блоков со свёртками 3×3 и MaxPool 2×2 один нейрон видит примерно 46×46 пикселей входа, а клетка после паддинга занимает почти весь кадр 224×224. Global Average Pooling просто усредняет такие локальные признаки, и сеть видит скорее «текстуру», чем форму клетки целиком. С пятым блоком поле становится около 94 пикселей, и форма уже попадает в признаки.

Вторым по важности оказался learning rate. При 3e-3 обучение нестабильное (val F1 0.706, early stopping на 39-й эпохе), при 3e-4 сеть за 50 эпох не успевает дойти до хорошего качества (test 0.745). 1e-3 из проверенных вариантов оптимальный.

С batch size 16 стало хуже (0.752 / 0.761), с 64 - немного лучше базы (0.802 / 0.835), хотя шагов оптимизатора при этом в два раза меньше. Скорее всего, при маленьком батче статистики BatchNorm считаются по 16 картинкам из 7 классов и сильно шумят.

Из функций активации GELU (0.836 / 0.846) и LeakyReLU (0.805 / 0.815) немного лучше ReLU. Из оптимизаторов AdamW с weight decay 1e-2 почти не отличается от Adam (0.809 / 0.799). У SGD val 0.742, а test 0.822, оценки расходятся в разные стороны. Dropout 0.2 (0.832 / 0.824) и 0.7 (0.736 / 0.807) по test оба лучше базового 0.5, что тоже похоже на шум. Меньший dropout логично помогает, так как сеть недообучена.

При этом в val и test всего по 156-157 картинок, одна ошибка меняет accuracy на 0.6%, а F1 по маленькому классу ещё сильнее. Разницы в пределах ±0.03-0.05 между запусками, которые отличаются «слабым» параметром (activation, optimizer, dropout), надёжными считать нельзя: val и test для них иногда меняются в разные стороны. Надёжно видны только крупные эффекты: глубина сети, слишком большой или маленький lr и аугментации.

В конце попробовал объединить три лучших по val изменения (5 блоков + GELU + dropout 0.2, запуск cnn_best). Комбинация оказалась хуже, чем просто 5 блоков: val 0.830, test 0.863. Val F1 у неё сильно прыгал, и early stopping остановил обучение на 27-й эпохе, когда сеть ещё не дошла до своего уровня. Эффекты отдельных изменений не складываются. По val лучшая конфигурация своей CNN - cnn_5blocks: ReLU, batch size 32, Adam, lr 1e-3, dropout 0.5, 5 блоков, без аугментаций.

5. ResNet18 и transfer learning

Берётся torchvision.models.resnet18 с весами IMAGENET1K_V1, последний слой fc (512 → 1000) заменяется на новый Linear(512, 7). Дальше четыре режима, которые отличаются только тем, какие слои обучаются. Замороженным параметрам ставится requires_grad = False, и в оптимизатор они не передаются.

Отдельно пришлось учесть BatchNorm: model.train() переключает все BN-слои в режим обучения, и они начинают обновлять running mean / var даже в замороженной части сети. Тогда «замороженный» backbone на самом деле меняется под маленький батч. Поэтому после model.train() BN-слои с замороженными весами возвращаются в eval (функция freeze_bn_stats в models.py).

режим	обучаются	заморожено	обучаемых параметров
resnet_fc	fc	conv1, bn1, layer1-layer4	3 591
resnet_layer4	layer4 + fc	conv1, bn1, layer1-layer3	8 397 319
resnet_layer3_4	layer3 + layer4 + fc	conv1, bn1, layer1-layer2	10 497 031
resnet_full	вся сеть	ничего	11 180 103

Параметры обучения одинаковые для всех четырёх режимов: те же splits/split.csv, без аугментаций (выбор из раздела 3), нормализация ImageNet, batch size 32, Adam, lr = 1e-3 для нового слоя fc и 1e-4 для размороженных слоёв backbone (предобученные веса не стоит сдвигать таким же большим шагом, как случайно инициализированный fc), weight decay 0, CosineAnnealingLR, не больше 50 эпох, early stopping с patience 10 по val macro F1. На test оценивается лучшая по val модель.

запуск	что изменено	best epoch	эпох	train F1*	val F1	test Acc	test macro F1
resnet_fc	FC	9	19	0.933	0.930	0.904	0.901
resnet_layer4	layer4 + FC	35	45	1.000	0.949	0.955	0.955
resnet_layer3_4	layer3-4 + FC	3	13	1.000	0.968	0.943	0.942
resnet_full	Full fine-tuning	8	18	1.000	0.969	0.924	0.924

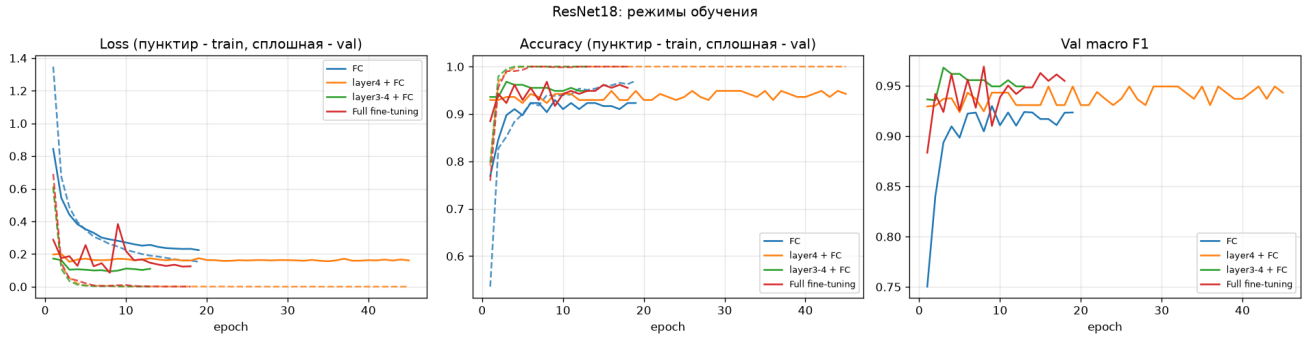


Рис. 6. Кривые обучения ResNet18 в четырёх режимах

Режим только с fc обучается быстро (лучшая эпоха 9) и без переобучения: train F1 0.933, val 0.930. Фактически это линейный классификатор поверх признаков ImageNet. Уже он намного лучше своей CNN (test 0.901 против 0.793 у базы и 0.886 у лучшей), но признаки под планктон не подстраиваются, и выше примерно 0.93 по val он не поднимается.

Разморозка layer4 даёт самый заметный прирост: val 0.949, test 0.955. Последние слои ResNet отвечают за самые «объектные» признаки ImageNet, и именно их полезнее всего переучить под клетки IFCB.

У layer3-4 и полной разморозки лучшие результаты по val (0.968 и 0.969), но по test они оказались хуже layer4 (0.942 и 0.924). Лучшая эпоха у них очень ранняя (3 и 8): уже через несколько эпох train F1 = 1.0, train loss ≈ 0.001-0.006, и дальше val F1 не растёт, поэтому через 10 эпох срабатывает early stopping. У полной разморозки val loss заметно скачет (например, с 0.09 до 0.38 между 8-й и 9-й эпохами), обучение 11 млн параметров на 730

картинках менее стабильное.

Порядок режимов по val и по test разный, поэтому по одному запуску нельзя уверенно сказать, что лучше: layer4, layer3-4 или полная разморозка. Ответ даёт повтор с другими seed в разделе 7.

Дополнительно: ResNet18 с аугментациями basic

Аугментации выбирались на своей CNN, а она, как выяснилось, недообучена. У ResNet18 ситуация другая: три из четырёх режимов запоминают train полностью. Поэтому отдельно (вне основного сравнения) обучил все четыре режима ещё раз с аугментациями basic, остальные параметры те же.

запуск	что изменено	best epoch	эпох	train F1*	val F1	test Acc	test macro F1
resnet_fc_aug	FC, basic	22	32	0.939	0.935	0.917	0.915
resnet_layer4_aug	layer4 + FC, basic	6	16	0.973	0.968	0.930	0.931
resnet_layer3_4_aug	layer3-4 + FC, basic	13	23	0.986	0.975	0.962	0.963
resnet_full_aug	Full fine-tuning, basic	22	32	0.992	0.974	0.917	0.917

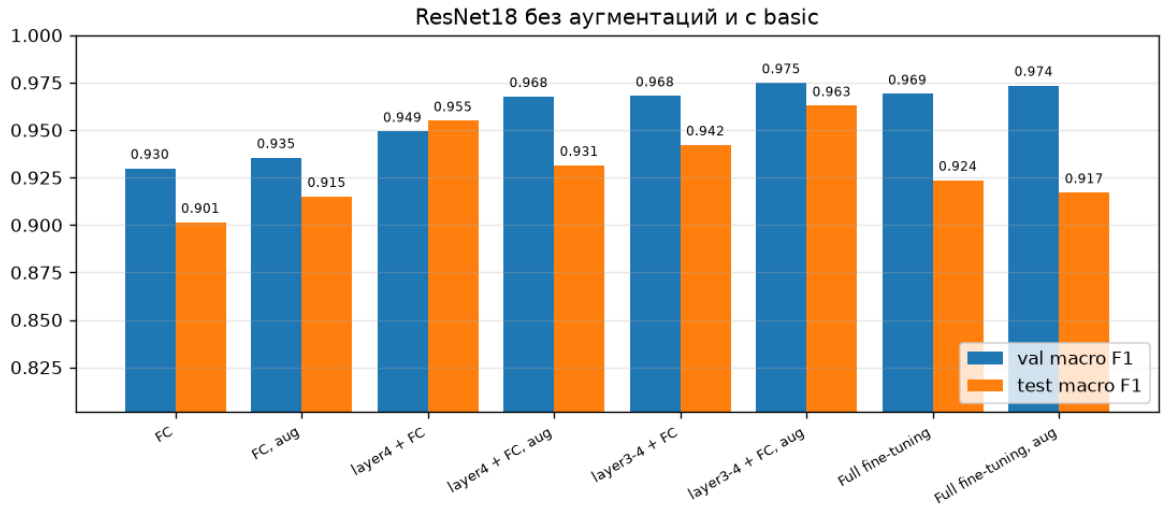


Рис. 6а. ResNet18 без аугментаций и с basic

По val аугментации помогли во всех четырёх режимах: fc 0.930 → 0.935, layer4 0.949 → 0.968, layer3-4 0.968 → 0.975, full 0.969 → 0.974. Train F1 у размороженных режимов стал 0.97-0.99 вместо 1.0, а лучшая эпоха в основном сдвинулась позже, похоже, аугментации работают как регуляризация и сеть меньше запоминает train. По test картина смешанная: fc и layer3-4 стали лучше (0.915 и 0.963), layer4 и full немного хуже (0.931 и 0.917). Разницы здесь 0.01-0.02, это 2-3 картинки из 157, так что надёжно можно сказать только одно: для предобученной сети, которая переобучается, аугментации basic не вредят и по val дают небольшой плюс, а для своей недообученной CNN они вредили сильно (-0.15 по val).

6. Метрики на test

Для каждой модели на test считаются Accuracy, Balanced Accuracy (средний recall по классам), Precision и Recall с macro-усреднением, Macro F1, F1 по каждому классу и confusion matrix. Метрики считаются через sklearn.metrics. Для всех остальных запусков те же метрики и графики лежат в results/runs/<запуск>/ (metrics.json, curves.png, cm_test.png).

модель	запуск	Accuracy	Balanced Acc	Precision (macro)	Recall (macro)	Macro F1
Custom CNN (база)	cnn_noaug	0.809	0.794	0.803	0.794	0.793
Custom CNN (лучшая)	cnn_5blocks	0.885	0.888	0.886	0.888	0.886
ResNet18 FC	resnet_fc	0.904	0.905	0.904	0.905	0.901
ResNet18 layer4 + FC	resnet_layer4	0.955	0.956	0.955	0.956	0.955
ResNet18 layer3-4 + FC	resnet_layer3_4	0.943	0.943	0.943	0.943	0.942
ResNet18 Full fine-tuning	resnet_full	0.924	0.925	0.924	0.925	0.924

F1 по классам на test

класс	Custom CNN (база)	Custom CNN (лучшая)	ResNet18 FC	ResNet18 layer4 + FC	ResNet18 layer3-4 + FC	ResNet18 Full fine-tuning
Cerataulina_flagellate	0.818	0.955	0.884	0.927	0.905	0.927
Chaetoceros_flagellate	0.844	0.955	0.889	0.936	0.913	0.936
Emiliania_huxleyi	0.933	0.977	1.000	1.000	0.977	1.000
Pleuronema_sp	0.571	0.889	0.865	0.971	0.971	0.971
Strombidium_oculatum	0.680	0.756	0.829	0.933	0.909	0.791
Strombidium_wulffi	0.739	0.708	0.880	0.917	0.920	0.840
dino_large1	0.964	0.963	0.963	1.000	1.000	1.000

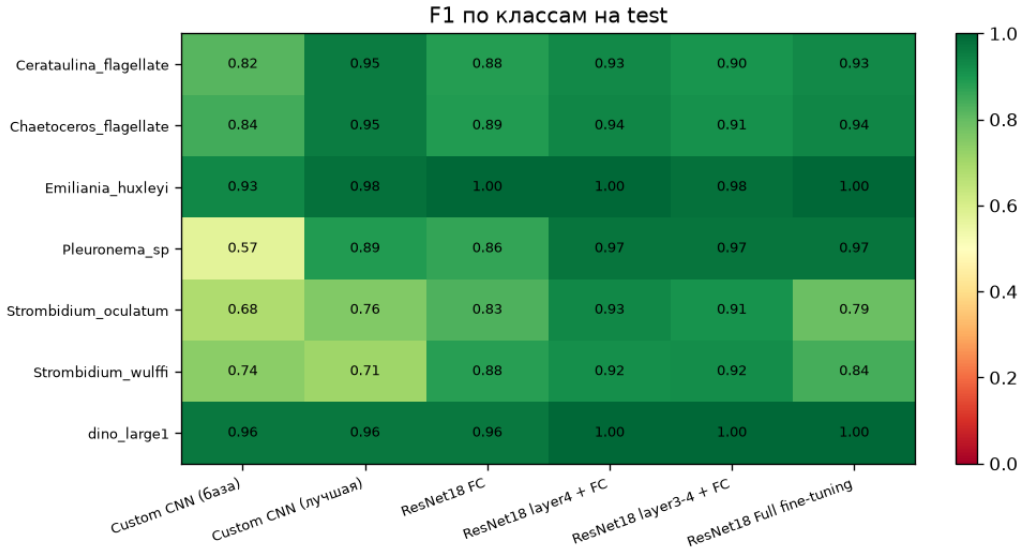


Рис. 7. F1 по классам для итоговых моделей

Хуже всего у всех моделей распознаются два вида Strombidium: у лучшей ResNet18 F1 около 0.92-0.93, у своей CNN 0.68-0.76. У базовой CNN (4 блока) ещё очень плохо с Pleuronema_sp (F1 0.571), это самый маленький класс (81 картинка в train). С пятым блоком и у ResNet18 он поднимается до 0.87-0.97. Лучше всего распознаются dino_large1 и Emiliania_huxleyi (0.96-1.00 почти у всех моделей): это округлые клетки с характерной текстурой, похожих на них классов среди выбранных нет.

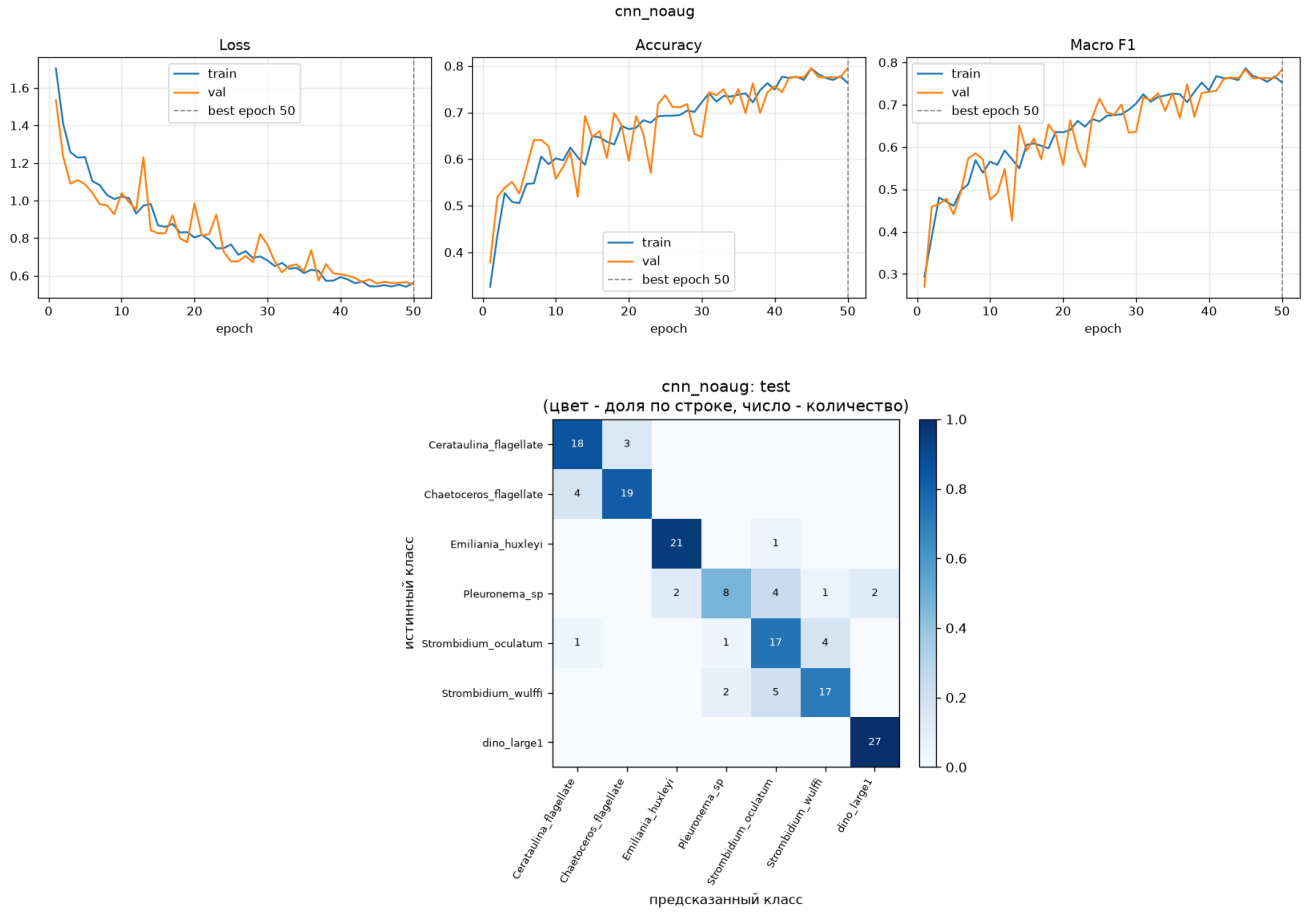
При этом лучшая CNN (5 блоков) на диатомеях с жгутиковыми (Cerataulina_flagellate и Chaetoceros_flagellate, F1 0.955) даже лучше всех ResNet18, а проигрывает в основном на Strombidium. Видимо, цепочки с мелкими клетками вокруг хорошо описываются текстурными признаками, а для двух видов Strombidium нужна именно форма.

Самые частые ошибки (по 3 на модель, test)

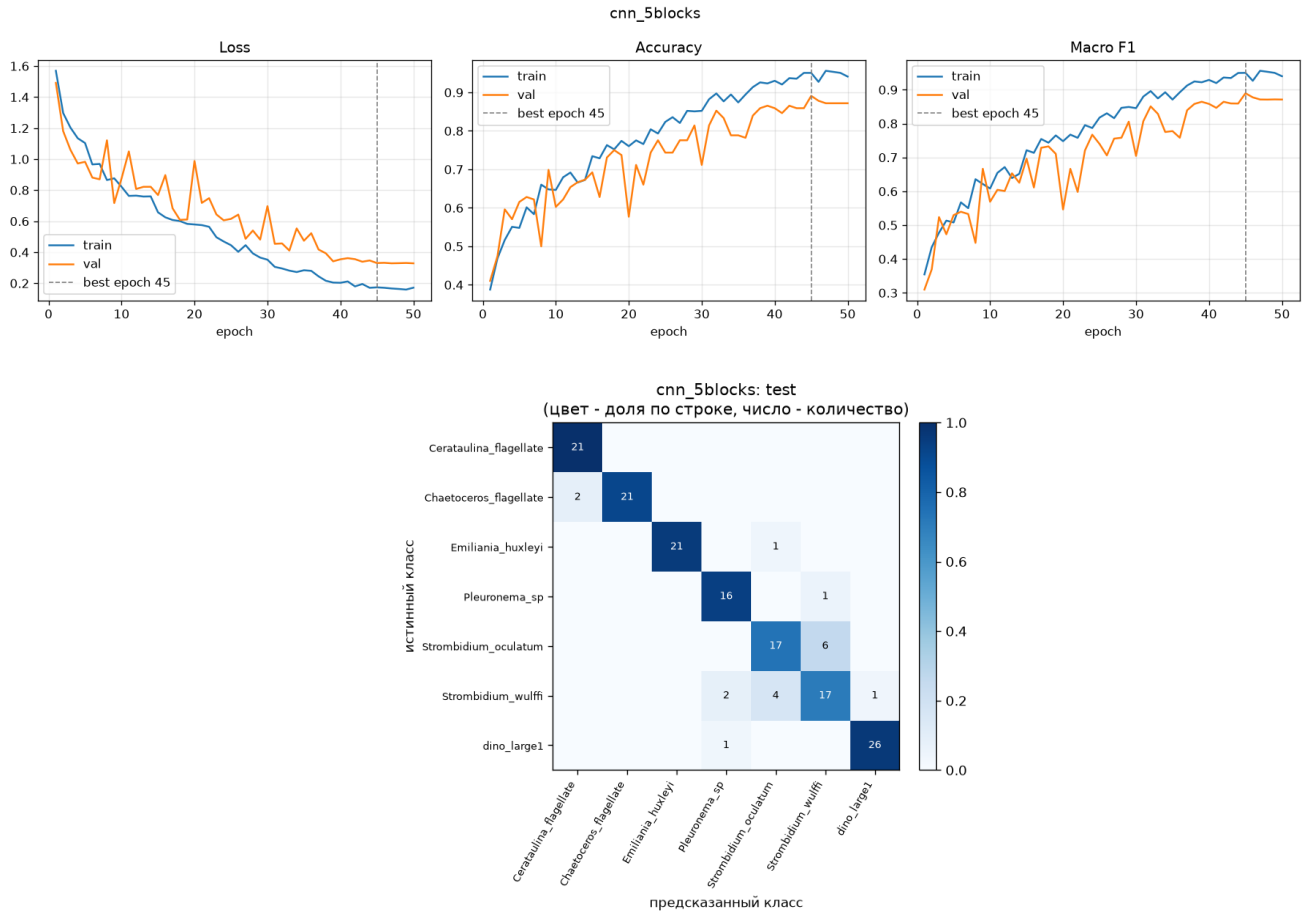
модель	истинный класс	предсказан как	сколько	доля класса
Custom CNN (база)	Strombidium_wulffi	Strombidium_oculatum	5	21%
Custom CNN (база)	Strombidium_oculatum	Strombidium_wulffi	4	17%
Custom CNN (база)	Pleuronema_sp	Strombidium_oculatum	4	24%
Custom CNN (лучшая)	Strombidium_oculatum	Strombidium_wulffi	6	26%
Custom CNN (лучшая)	Strombidium_wulffi	Strombidium_oculatum	4	17%
Custom CNN (лучшая)	Strombidium_wulffi	Pleuronema_sp	2	8%
ResNet18 FC	Strombidium_oculatum	Strombidium_wulffi	4	17%
ResNet18 FC	Chaetoceros_flagellate	Cerataulina_flagellate	3	13%
ResNet18 FC	Strombidium_oculatum	Pleuronema_sp	2	9%
ResNet18 layer4 + FC	Strombidium_oculatum	Strombidium_wulffi	2	9%
ResNet18 layer4 + FC	Cerataulina_flagellate	Chaetoceros_flagellate	2	10%
ResNet18 layer4 + FC	Strombidium_wulffi	Strombidium_oculatum	1	4%
ResNet18 layer3-4 + FC	Strombidium_oculatum	Strombidium_wulffi	3	13%
ResNet18 layer3-4 + FC	Chaetoceros_flagellate	Cerataulina_flagellate	2	9%
ResNet18 layer3-4 + FC	Cerataulina_flagellate	Chaetoceros_flagellate	2	10%
ResNet18 Full fine-tuning	Strombidium_oculatum	Strombidium_wulffi	5	22%
ResNet18 Full fine-tuning	Strombidium_wulffi	Strombidium_oculatum	3	12%
ResNet18 Full fine-tuning	Cerataulina_flagellate	Chaetoceros_flagellate	2	10%

У всех моделей одни и те же ошибки. Чаще всего путаются два вида Strombidium (oculatum и wulffi): это один род, у обоих коническая клетка с венчиком ресничек, и различаются они в основном формой и пропорциями. Вторая пара - Cerataulina_flagellate и Chaetoceros_flagellate: оба класса - цепочки диатомей с прикреплёнными мелкими жгутиковыми, а на снимке 224×224 сама цепочка часто видна плохо. Иногда в Pleuronema_sp уходят инфузории Strombidium, это тоже овальные клетки с ресничками. dino_large1 и Emiliania_huxleyi почти не путаются ни с чем.

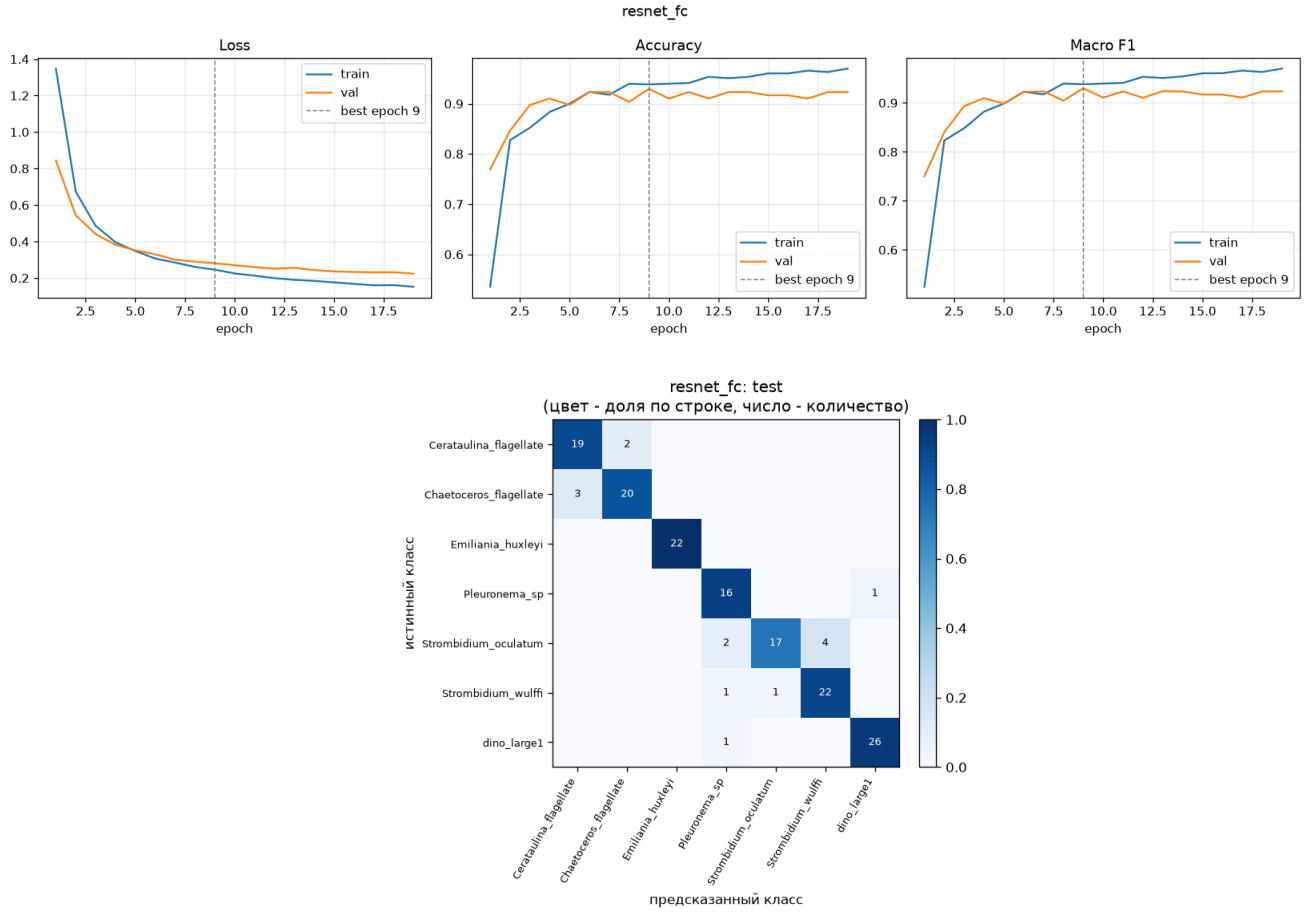
Confusion matrix и кривые обучения итоговых моделей



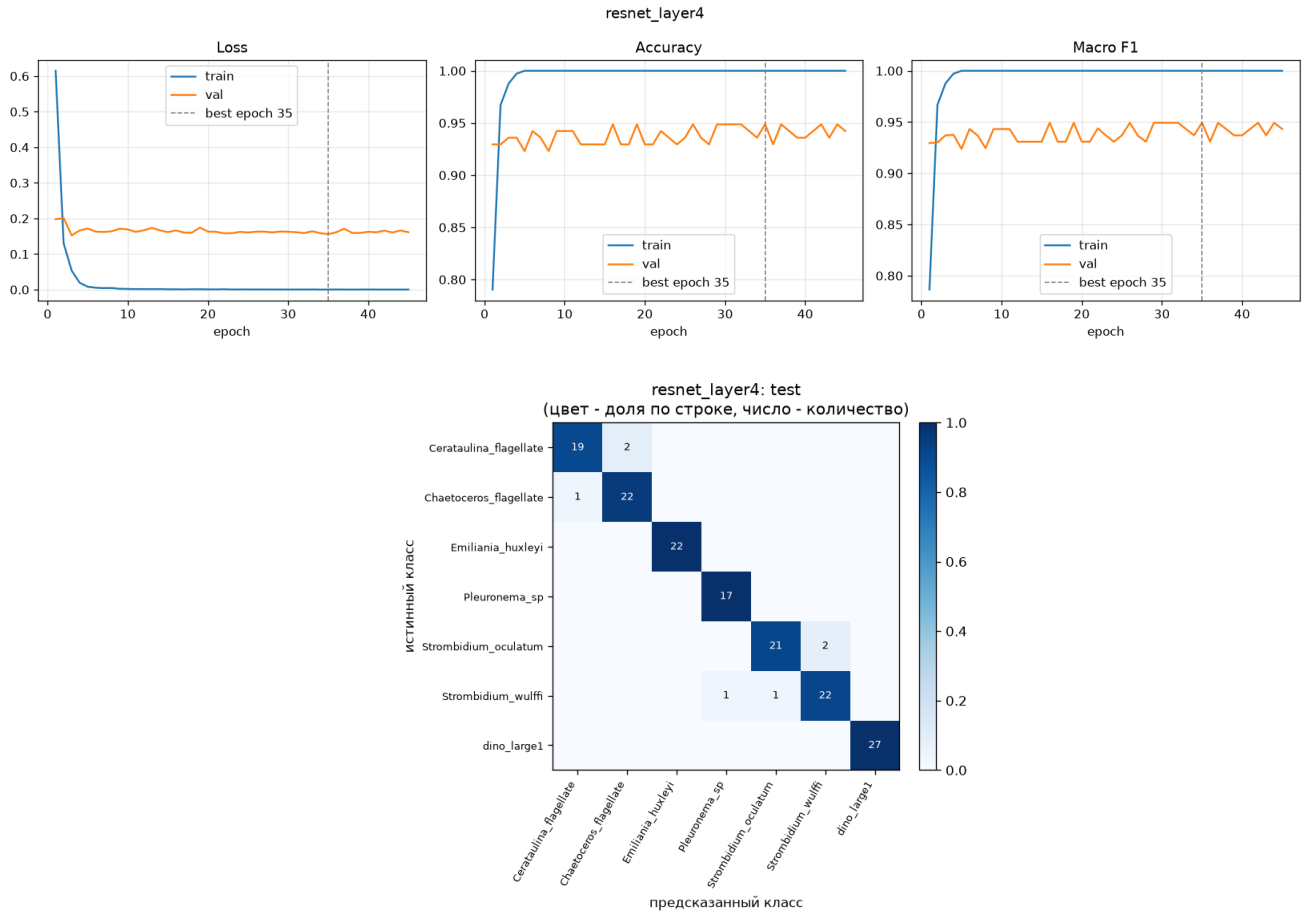
Custom CNN (база): loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test



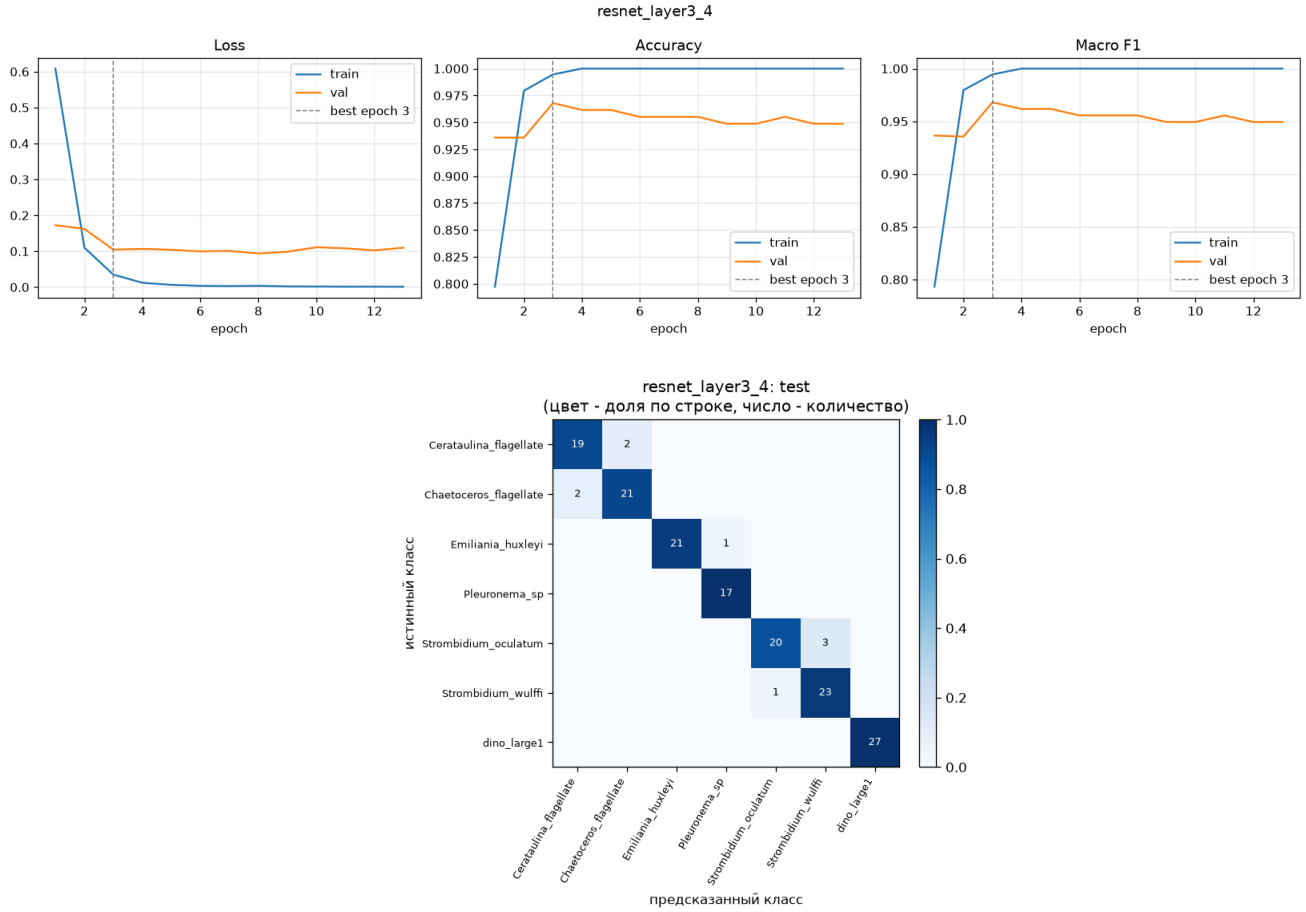
Custom CNN (лучшая): loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test



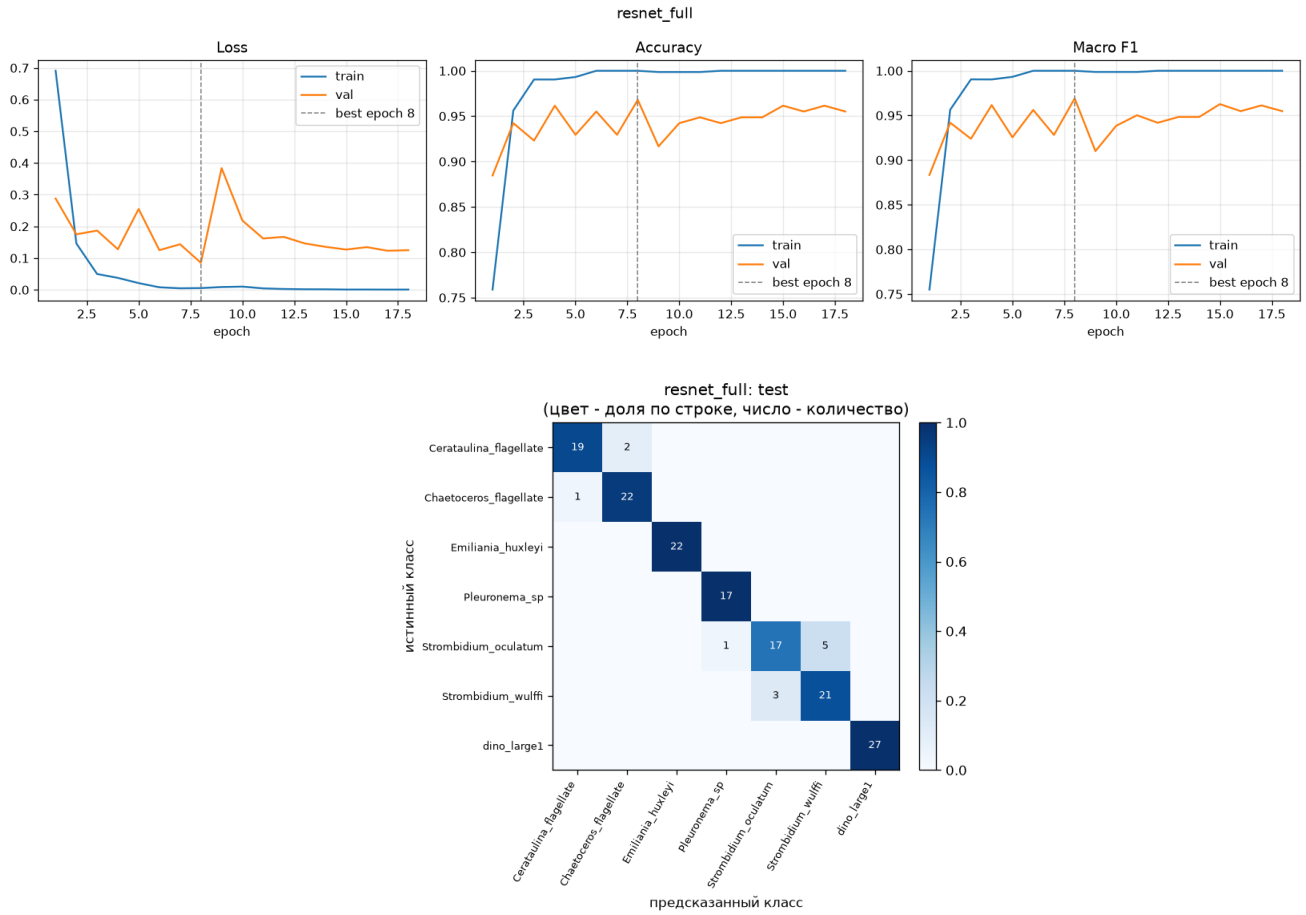
ResNet18 FC: loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test



ResNet18 layer4 + FC: loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test



ResNet18 layer3-4 + FC: loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test



ResNet18 Full fine-tuning: loss, accuracy и macro F1 по эпохам (пунктир - лучшая эпоха) и confusion matrix на test

7. Сравнение моделей

Итоговая таблица в формате из задания. Для своей CNN две строки. Базовая (cnn_noaug) обучена ровно в тех же условиях, что и ResNet18: те же выборки, без аугментаций, batch size 32, Adam, косинусное расписание, не больше 50 эпох, тот же early stopping. Лучшая (cnn_5blocks) выбрана по val после исследования параметров.

Model	Trainable layers	Trainable params	Aug	Batch size	Activation	Dropout	Optimizer	LR	Best epoch	Accuracy	Macro F1
Custom CNN (база)	все (с нуля)	456 487	none	32	relu	0.5	adam	0.001	50	0.809	0.793
Custom CNN (лучшая)	все (с нуля)	1 702 695	none	32	relu	0.5	adam	0.001	45	0.885	0.886
ResNet18	FC	3 591	none	32	relu	-	adam	0.001	9	0.904	0.901
ResNet18	layer4 + FC	8 397 319	none	32	relu	-	adam	0.001 / 0.0001	35	0.955	0.955
ResNet18	layer3-4 + FC	10 497 031	none	32	relu	-	adam	0.001 / 0.0001	3	0.943	0.942
ResNet18	Full fine-tuning	11 180 103	none	32	relu	-	adam	0.001 / 0.0001	8	0.924	0.924

Общее для всех строк: разбиение splits/split.csv (730 / 156 / 157), seed 42, вход 3×224×224 после паддинга до квадрата, CrossEntropyLoss без весов классов, CosineAnnealingLR на 50 эпох, early stopping с patience 10 по val macro F1, на test - лучшая по val модель. Нормализация для CNN - по train (mean 0.664, std 0.164), для ResNet18 - ImageNet. LR у ResNet18 записан как «lr fc / lr backbone», для режима FC обучается только fc с lr 1e-3. Аугментаций в итоговых моделях нет. Обучение на Apple M2 (MPS).

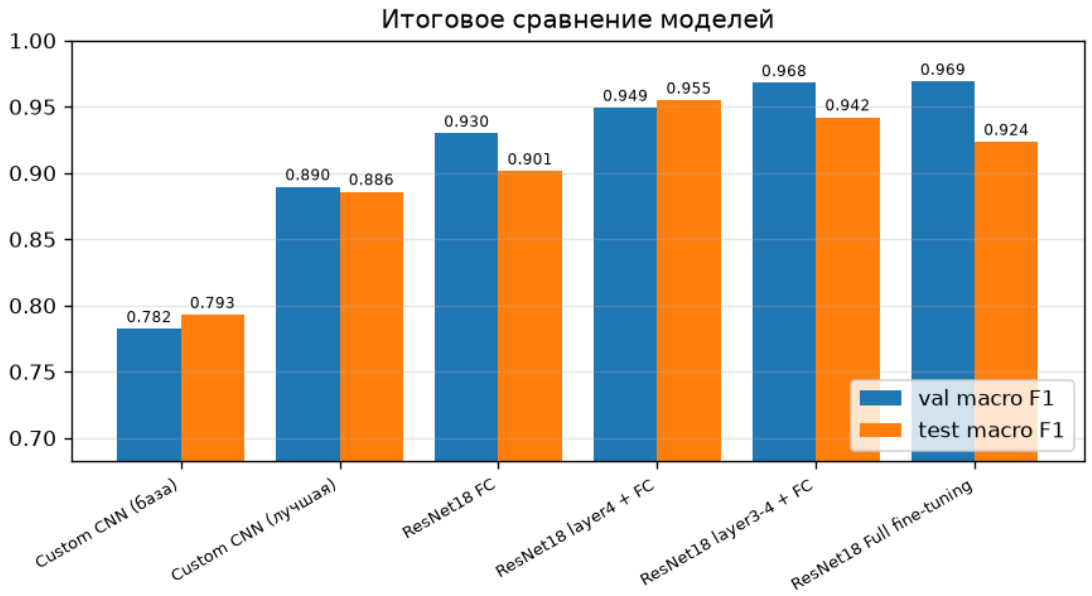


Рис. 8. Macro F1 итоговых моделей на val и test

Переобучение

модель	train F1*	val F1	test F1	train loss*	val loss	test loss
Custom CNN (база)	0.809	0.782	0.793	0.462	0.555	0.492
Custom CNN (лучшая)	0.969	0.890	0.886	0.115	0.330	0.267
ResNet18 FC	0.933	0.930	0.901	0.230	0.281	0.272
ResNet18 layer4 + FC	1.000	0.949	0.955	0.000	0.156	0.131
ResNet18 layer3-4 + FC	1.000	0.968	0.942	0.006	0.104	0.161
ResNet18 Full fine-tuning	1.000	0.969	0.924	0.001	0.085	0.182

* train F1 и train loss - на train без аугментаций, лучшая по val модель.

Своя CNN с 4 блоками не переобучается совсем: train F1 0.809, val 0.782, test 0.793, loss на train и val почти одинаковый. Это недообучение, а не переобучение: модели не хватает шагов обучения и рецептивного поля. У CNN с 5 блоками появляется разрыв (train 0.969, val 0.890, train loss 0.115 против 0.330 на val). Это уже переобучение, но умеренное.

ResNet18 только с fc тоже не переобучается (0.933 / 0.930). Все режимы с размороженными слоями полностью запоминают train: F1 = 1.000, train loss 0.000-0.006 при val loss 0.09-0.16 и test loss 0.13-0.18. Разрыв по loss большой, но на val F1 это не сказывается: кривые val не падают, а держатся на одном уровне. Помогают предобученные веса, маленький lr для backbone (1e-4) и early stopping, который берёт раннюю эпоху. Аугментации (раздел 5) этот разрыв уменьшают.

Повторы с другими seed

Так как val и test маленькие, итоговые модели я обучил ещё два раза с seed 1 и 2 (меняется инициализация fc / всей CNN и порядок батчей, разбиение то же). В таблице среднее ± стандартное отклонение по трём запускам (seed 42, 1, 2).

модель	запусков	val F1	test Accuracy	test Macro F1
Custom CNN (база)	3	0.797 ± 0.023	0.832 ± 0.021	0.825 ± 0.028
Custom CNN (лучшая)	3	0.826 ± 0.077	0.802 ± 0.122	0.796 ± 0.132
ResNet18 FC	3	0.929 ± 0.001	0.921 ± 0.019	0.920 ± 0.020
ResNet18 layer4 + FC	3	0.956 ± 0.017	0.949 ± 0.011	0.948 ± 0.012
ResNet18 layer3-4 + FC	3	0.966 ± 0.003	0.941 ± 0.004	0.940 ± 0.003
ResNet18 Full fine-tuning	3	0.958 ± 0.013	0.930 ± 0.017	0.930 ± 0.018

По трём seed картина понятнее, чем по одному запуску. ResNet18 в любом режиме стабильно лучше своей CNN примерно на 0.1-0.12 по test macro F1 (0.92-0.95 против 0.80-0.83). Среди режимов ResNet18 заметно хуже остальных только fc (0.920 ± 0.020). layer4 (0.948 ± 0.012) и layer3-4 (0.940 ± 0.003) по test отличаются меньше, чем на одно стандартное отклонение, а по val впереди layer3-4 (0.966 против 0.956). Полная разморозка по test не лучше частичной (0.930 ± 0.018), а по разбросу между seed она самая нестабильная из размороженных режимов. Самый стабильный режим - layer3-4: у него и val, и test почти не зависят от seed.

Хорошо видно и то, насколько сильно на маленьких выборках влияет случайность. У одной и той же базовой CNN test macro F1 по трём seed - 0.793, 0.843 и 0.839, то есть разница 0.05 получается только из-за инициализации и порядка батчей. Это больше, чем эффект многих параметров из раздела 4. Поэтому выводы раздела 4 про activation, optimizer и dropout я считаю предварительными. У CNN с 5 блоками разброс ещё больше (0.886, 0.645, 0.858), причина разобрана ниже.

Early stopping у своей CNN

При повторах выяснилось, что у CNN с 5 блоками early stopping иногда срабатывает слишком рано. На маленьком val случайный всплеск F1 в начале обучения становится «лучшим результатом», и за следующие 10 эпох его не удаётся превзойти, хотя сеть ещё учится. Например, при seed 1 val F1 на 9-й эпохе был 0.74 при train F1 всего 0.58, и обучение остановилось на 19-й эпохе. Чтобы понять, сколько это стоит, обучил те же seed без early stopping (patience = 50, то есть все 50 эпох; лучшая по val модель всё равно сохраняется и идёт на test). У seed 42 early stopping не сработал, он и так обучался 50 эпох.

seed	с early stopping: best / эпох	test F1	без early stopping: best / эпох	test F1
1	9 / 19	0.645	32 / 50	0.856
2	26 / 36	0.858	46 / 50	0.905

Без early stopping CNN с 5 блоками по трём seed даёт test macro F1 0.882 ± 0.025 (val 0.886 ± 0.008), а не 0.796 ± 0.132. У базовой CNN с 4 блоками early stopping ни разу не сработал, у неё 0.825 ± 0.028. Значит, выигрыш от пятого блока реальный, около +0.06 по test и +0.09 по val, а огромный разброс был артефактом критерия остановки. Для ResNet18 такой проблемы нет: он сходится за несколько эпох, и ранняя остановка там оправдана.

Получается, что для сети, которая медленно учится с нуля, и val из 156 картинок patience 10 по macro F1 слишком мало. Лучше останавливаться по val loss, который шумит меньше, или брать patience 15-20. В основной таблице я оставил протокол, зафиксированный заранее, одинаковый для всех моделей.

8. Выводы

1. Насколько хорошо собственная CNN работает при заданном количестве изображений?

Средне. Базовая CNN из задания (4 блока) даёт test macro F1 0.825 ± 0.028 по трём seed (accuracy около 0.83). С пятым блоком, если не мешает early stopping, получается 0.882 ± 0.025. Для 7 классов это намного выше случайного уровня (0.14), но всё равно ошибается примерно на каждой 6-8-й картинке, а виды Strombidium и Pleuronema_sp распознаются плохо (F1 0.57-0.76). Главная проблема - недообучение: 730 картинок дают всего около 23 шагов оптимизатора за эпоху, и за 50 эпох сеть с нуля не успевает сойтись (лучшая эпоха почти всегда одна из последних). У базовой архитектуры к тому же маленькое рецептивное поле.

2. Даёт ли transfer learning преимущество перед обучением CNN с нуля?

Да, и это самый большой эффект во всей работе. Даже ResNet18 с полностью замороженным backbone (обучается только fc, 3 591 параметр) даёт 0.920 ± 0.020 против 0.825 у базовой CNN и 0.882 у лучшей. Лучшие режимы дообучения дают 0.94-0.95. Кроме качества, ResNet18 сходится в разы быстрее (лучшая эпоха от 3 до 35, у CNN - 30-50) и в целом меньше зависит от seed. Низкоуровневые признаки, выученные на ImageNet (края, текстуры, контуры), подходят и для снимков планктона, а выучить их с нуля по 730 картинкам не получается.

3. Как глубина fine-tuning влияет на качество ResNet18?

Зависимость немонотонная. Разморозка layer4 даёт прирост около +0.03 по сравнению с одним fc (0.948 против 0.920 по test, 0.956 против 0.929 по val). Разморозка ещё и layer3 по test даёт примерно то же (0.940), по val чуть лучше (0.966), и это самый стабильный режим. Полная разморозка не добавляет ничего (0.930 по test, 0.958 по val) и сильнее зависит от seed. То есть подстраивать под планктон стоит верхние, более «объектные» слои, а нижние (conv1, layer1-2) универсальные, и их лучше не трогать.

4. Есть ли смысл размораживать всю сеть при данном количестве данных?

При 100-200 картинках на класс - нет. Качество не выше, чем при частичной разморозке, а обучаемых параметров 11.2 млн вместо 8.4 млн (layer4) или 10.5 млн (layer3-4). Сеть за несколько эпох полностью запоминает train (F1 = 1.0, loss ≈ 0.001), val loss скачет от эпохи к эпохе, результат сильнее зависит от seed, а обучение идёт дольше. Оптимально размораживать layer4 или layer3-4.

5. Как изменение параметров собственной CNN влияет на результат?

Сильнее всего влияют архитектура и learning rate. Пятый свёрточный блок даёт +0.06 по test и +0.09 по val в среднем по трём seed за счёт большего рецептивного поля. lr 3e-3 и 3e-4 оба хуже, чем 1e-3 (на 0.05-0.13 по test). Аугментации в моём случае ухудшили результат на 0.1-0.17, частично из-за поворотов на 90° (см. раздел 3). Batch size 16 хуже базового, 64 немного лучше. Функция активации (GELU и LeakyReLU немного лучше ReLU), оптимизатор (AdamW ≈ Adam, SGD неоднозначно) и dropout меняют результат на ±0.03-0.05. Это сравнимо с разбросом от одного только seed (0.793-0.843 у базы), так что надёжного вывода по ним сделать нельзя. Объединение трёх лучших изменений результат не улучшило.

6. Какие классы классифицируются хуже всего?

Strombidium_oculatum и Strombidium_wulffi - у всех моделей. У лучшей ResNet18 F1 по ним около 0.92-0.93, у своей CNN - 0.68-0.76. У базовой CNN ещё очень плохо распознаётся Pleuronema_sp (F1 0.571), самый маленький класс. Лучше всего распознаются dino_large1 и Emiliania_huxleyi (F1 0.96-1.00).

7. Какие классы чаще всего путаются между собой?

Strombidium_oculatum и Strombidium_wulffi: это один род с очень похожей формой. У CNN до 21-26% картинок одного вида уходит в другой, у ResNet18 - 9-22%. Вторая пара - Cerataulina_flagellate и Chaetoceros_flagellate, обе - цепочки диатомей с прикреплёнными жгутиковыми. Реже инфузории Strombidium уходят в Pleuronema_sp.

8. Наблюдаются ли признаки переобучения?

У своей CNN с 4 блоками - нет, наоборот, недообучение: train F1 почти равен val, train и val loss одинаковые, лучшая эпоха последняя. С 5 блоками появляется умеренный разрыв (train 0.97, val 0.89). У ResNet18 с размороженными слоями переобучение явное: через несколько эпох train F1 = 1.0 и train loss почти 0, а val loss в десятки раз больше. Но качество на val при этом не падает, а держится на плато. Помогают предобученные веса, маленький lr для backbone и early stopping, который берёт раннюю эпоху. Аугментации у ResNet18 немного уменьшают этот разрыв и поднимают val. Режим только с fc не переобучается.

9. Какая конфигурация показала лучший результат и почему?

ResNet18 с размороженными верхними блоками. По test в среднем по трём seed лучше всех layer4 + fc (macro F1 0.948 ± 0.012, accuracy 0.949), по val - layer3-4 + fc (0.966 ± 0.003), и он же самый стабильный. Разница между ними в пределах шума, но обе лучше и одного fc, и полной разморозки. Думаю, дело в том, что низкоуровневые признаки ImageNet подходят и для планктона и их лучше не трогать, а верхние слои подстраиваются под формы и текстуры клеток. Параметров для такой подстройки хватает, а обучение стабильнее, чем при разморозке всей сети. Из своих CNN лучшая - с 5 блоками (0.886 на seed 42), но до ResNet18 ей далеко.

Приложение. Все запуски

Все запуски с основными параметрами и метриками (полная таблица - results/summary.csv). train F1* - на train без аугментаций, как в разделе 3.

запуск	aug	act	блоков	bs	opt	lr	drop	best ep	train F1*	val F1	test Acc	test Bal.Acc	test P	test R	test F1	мин
cnn_5blocks	none	relu	5	32	adam	0.001	0.5	45	0.969	0.890	0.885	0.888	0.886	0.888	0.886	5.6
cnn_adamw	none	relu	4	32	adamw	0.001	0.5	45	0.815	0.809	0.809	0.798	0.806	0.798	0.799	5.6
cnn_aug	basic	relu	4	32	adam	0.001	0.5	47	0.664	0.634	0.726	0.703	0.775	0.703	0.693	6.7
cnn_aug_flip	flip	relu	4	32	adam	0.001	0.5	40	0.743	0.695	0.790	0.768	0.816	0.768	0.762	5.2
cnn_aug_strong	strong	relu	4	32	adam	0.001	0.5	32	0.586	0.595	0.688	0.658	0.592	0.658	0.622	5.4
cnn_best	none	gelu	5	32	adam	0.001	0.2	17	0.867	0.830	0.866	0.862	0.868	0.862	0.863	3.1
cnn_bs16	none	relu	4	16	adam	0.001	0.5	44	0.772	0.752	0.777	0.763	0.777	0.763	0.761	5.8
cnn_bs64	none	relu	4	64	adam	0.001	0.5	50	0.841	0.802	0.841	0.832	0.842	0.832	0.835	5.9
cnn_drop0.2	none	relu	4	32	adam	0.001	0.2	50	0.845	0.832	0.834	0.823	0.833	0.823	0.824	5.0
cnn_drop0.7	none	relu	4	32	adam	0.001	0.7	43	0.774	0.736	0.815	0.803	0.819	0.803	0.807	4.9
cnn_gelu	none	gelu	4	32	adam	0.001	0.5	43	0.869	0.836	0.847	0.846	0.851	0.846	0.846	6.0
cnn_leaky_relu	none	leaky_relu	4	32	adam	0.001	0.5	43	0.819	0.805	0.822	0.813	0.822	0.813	0.815	5.7
cnn_lr3e-3	none	relu	4	32	adam	0.003	0.5	29	0.676	0.706	0.701	0.677	0.696	0.677	0.662	4.7
cnn_lr3e-4	none	relu	4	32	adam	0.0003	0.5	41	0.795	0.805	0.764	0.748	0.761	0.748	0.745	5.9
cnn_noaug	none	relu	4	32	adam	0.001	0.5	50	0.809	0.782	0.809	0.794	0.803	0.794	0.793	6.3
cnn_sgd	none	relu	4	32	sgd	0.01	0.5	42	0.800	0.742	0.834	0.821	0.837	0.821	0.822	5.1
resnet_fc	none	relu		32	adam	0.001		9	0.933	0.930	0.904	0.905	0.904	0.905	0.901	1.2
resnet_fc_aug	basic	relu		32	adam	0.001		22	0.939	0.935	0.917	0.915	0.917	0.915	0.915	2.0
resnet_full	none	relu		32	adam	0.001 / 0.0001		8	1.000	0.969	0.924	0.925	0.924	0.925	0.924	3.1
resnet_full_aug	basic	relu		32	adam	0.001 / 0.0001		22	0.992	0.974	0.917	0.919	0.930	0.919	0.917	4.8
resnet_layer3_4	none	relu		32	adam	0.001 / 0.0001		3	1.000	0.968	0.943	0.943	0.943	0.943	0.942	1.5
resnet_layer3_4_aug	basic	relu		32	adam	0.001 / 0.0001		13	0.986	0.975	0.962	0.964	0.963	0.964	0.963	2.2
resnet_layer4	none	relu		32	adam	0.001 / 0.0001		35	1.000	0.949	0.955	0.956	0.955	0.956	0.955	4.0

запуск	aug	act	блоков	bs	opt	lr	drop	best ep	train F1*	val F1	test Acc	test Bal.Acc	test P	test R	test F1	мин
resnet_layer4_aug	basic	relu		32	adam	0.001 / 0.0001		6	0.973	0.968	0.930	0.930	0.933	0.930	0.931	1.2

Воспроизведение

- pip install -r requirements.txt, затем hf auth login (карточка gated-датасета открывается только с токеном);
- python prepare_data.py - подсчёт классов, выгрузка картинок в data/, разбиение в splits/split.csv;
- bash run_experiments.sh - все запуски по очереди, результаты в results/runs/;
- python summarize.py - сводная таблица results/summary.csv, графики results/figures/ и выбор лучшей CNN по val;
- python run_seeds.py, затем ещё раз python summarize.py - повторы итоговых моделей с seed 1 и 2;
- python make_report.py - этот отчёт.
- На MPS часть операций недетерминирована, поэтому при повторном запуске числа могут немного отличаться (в пределах нескольких сотых по F1).